System Requirements Document (SRD)

Theoretical Field Service Management System

Created By: Le'Shawn Sears

1. Introduction

1.1. Purpose:

The purpose of this Software Requirements Document (SRD) is to define the functional and non-functional requirements for the development of a Field Service Management (FSM) software tailored for technicians and contractors working in fields such as HVAC and carpentry. The FSM software aims to streamline operational tasks, enhance workforce efficiency, and ensure compliance with American standards related to field operations work. This document will guide the development, testing, and implementation of the software to meet these objectives, providing a comprehensive solution for job scheduling, real-time tracking, inventory management, invoicing, performance reporting, offline access, and more.

1.2. Scope:

The scope of the FSM software encompasses the development and implementation of a comprehensive system designed to manage and streamline field service operations. The software will provide functionalities including:

- User Authentication and Role Management: Secure access control and permission management for various user roles, including technicians, supervisors, and administrators.
- Job Scheduling and Dispatch: Tools for creating, modifying, and optimizing job schedules for field technicians.
- Real-time GPS tracking: to monitor technician locations and job status with GPS.
- Invoicing and Payment Processing: Capabilities for generating invoices, processing payments, and managing financial transactions.
- Inventory Management: Tracking and managing inventory levels and supplies used in field operations.
- Integration with Third-Party Tools: Interfaces for integrating with other systems such as ERP and CRM for seamless data exchange
- Performance Reporting: Reporting tools to analyze and assess performance metrics and service outcomes.

- Customer Feedback and Ratings: Mechanisms for capturing and analyzing customer feedback and ratings.
- Offline Access: Functionality to enable access to critical features and data even when disconnected from the internet.
- Data Security and Compliance: Measures to protect sensitive data and ensure compliance with relevant regulations, including OSHA and DOT.

1.3. Definitions, Acronyms, and Abbreviations:

- **FSM**: Field Service Management
- **ERP**: Enterprise Resource Planning
- **CRM**: Customer Relationship Management
- **GPS**: Global Positioning System
- **SRD**: System Requirements Document
- **API**: Application Programming Interface
- **OSHA**: Occupational Safety and Health Administration
- **DOT**: Department of Transportation
- **MFA**: Multi-Factor Authentication

1.4. Overview:

The SRD is organized into several key sections to provide a structured and comprehensive view of the system requirements. The document includes:

- Overall Description: Provides a high-level overview of the system, including its context, major functionalities, and user characteristics.
    - Specific Requirements: Details the functional and non-functional requirements of the system, including inputs, processing, and outputs for each requirement.
    - External Interface Requirements: Specifies how the system will interact with external hardware, software, and communication interfaces.
    - System Features: Breaks down major system features and their detailed descriptions.
    - Other Non-Functional Requirements: Addresses additional non-functional aspects such as performance, security, and compliance.
    - Appendices: Includes supplementary information such as glossaries, analysis models, and supporting documentation.

2. Overall Description

2.1. Product Perspective:

The FSM software is tailored for managing field service operations primarily in technician-driven and contractor-based industries such as HVAC, carpentry, plumbing, and electrical work. The software integrates with ERP and CRM systems to ensure seamless data flow between field operations and back-office management. The FSM system supports real-time tracking, job scheduling, inventory management, and technician performance monitoring, making it an essential tool for organizations focused on field services. Integration with third-party tools like QuickBooks for financials or Google Maps for location services are essential to maintain operational efficiency.

2.2. Product Functions:

The FSM software includes key functionalities tailored to the needs of technicians and contractors:

- User Authentication and Role Management
  - Secure login using MFA and management of user roles to control access to features based on the user's role (e.g., technician, contractor, administrator).
- Job Scheduling and Dispatch
  - Automated assignment of jobs based on technician location, availability, skillset, and compliance with OSHA safety standards.
- Real-Time GPS Tracking
  - Provides real-time visibility into technician locations to optimize routing and ensure compliance with DOT guidelines.
- Inventory Management
  - Management of inventory specific to HVAC, carpentry, and other trades, including tracking and inspection of supplies and tools needed for field of work.
- Invoicing and Payment Processing
  - Capabilities for generating invoices, processing payments, and managing financial transactions streamlining financial operations for service-based businesses.
- Performance Reporting
  - Reporting tools to analyze metrics such as job completion times, technician efficiency, and service outcomes, tailored to industry-specific needs.
- Offline Access
  - Functionality for technicians to access job information and update statuses while offline, with synchronization upon reconnection.

2.3. User Classes and Characteristics:

- Technicians and Contractors:
  - Characteristics: Field workers performing tasks related to HVAC, carpentry, and other trades.
  - Needs: Access to job details, real-time updates, GPS navigation, offline access to job information.
- Operation Managers:
  - Characteristics: Responsible for dispatching and overseeing technician performance, ensuring compliance with regulations like OSHA.
  - Needs: Job assignments, status updates, invoicing, and payment processing.
- Administrators:
  - Characteristics: Individuals responsible for managing system configurations, user roles, and overall operations.
  - Needs: System management tools, role management, access to performance reports, and configuration settings.

2.4. Operating Environment:

The FSM software will operate in the following environments:

- **Hardware**:
  - Desktop and mobile devices, including those with GPS capabilities for field technicians.
- **Software**:
  - Web browsers (e.g., Chrome, Firefox) for accessing the application.
  - Mobile operating systems (e.g., iOS, Android) for mobile access.
- **Network:**
  - Requires internet connectivity for full functionality, with offline capabilities for field technicians.
- **Databases:**
  - Utilizes relational databases (e.g., PostgreSQL, MySQL) for storing job-related data, inventory information, and user details.

2.5. Design and Implementation Constraints:

- Regulations and Standards:

    - Compliance with OSHA (Occupational Safety and Health Administration) regulations for workplace safety.
    - Adherence to DOT (Department of Transportation) regulations for vehicle use and transportation safety.
- Development Tools:
    - Developed using Java (backend) and JavaScript (frontend), with frameworks such as Spring Boot and React.
- Third-Party Libraries:
    - Use APIs to integrate with libraries and tools such as Salesforce, QuickBooks, Shopify, and Google Maps for GPS tracking, payment processing, and inventory management.

2.6. Assumptions and Dependencies:

- **Assumptions**:
    - Users will have internet access for most features, with offline access available as a backup.
    - Integration with **American standard-compliant** tools and databases.
- Dependencies:
    - Dependence on third-party services for GPS tracking and payment processing.
    - Compliance with ERP systems as well as OSHA and DOT regulations as part of the system's operational framework.

3. Specific Requirements

3.1. Functional Requirements

- **User Authentication and Role Management**
    - Description: The system must provide secure user authentication and manage role-based access for different user roles such as technicians, managers, and administrators. Implement using DUO Security's SDK or API for MFA integration.
    - Inputs: User credentials, MFA token from DUO Security.
    - Processing: Validate user credentials and MFA token. Assign roles and permissions based on user profile.
    - Outputs: Access granted/denied messages, user dashboard views.
    - Related Test Cases: Verify the effectiveness of user authentication and MFA integration, including testing scenarios where MFA tokens are valid or invalid.

- Pre-Conditions: User roles defined; DUO Security integration configured; test accounts created; access control policies configured
- Expected Output: Users can access only features allowed by their role; MFA is required for login, and unauthorized actions result in access denied messages.

- **Technologies and Tools:**
  - Authentication System: DUO Security (for MFA)
  - Programming Languages: Java, Python, JavaScript (Node.js)
  - Frameworks: Spring Boot (Java), Django (Python), Express.js (Node.js)
  - Frontend Libraries: React, Angular, Vue.js

- **Job Scheduling and Dispatch**
  - Description: Requirements for creating, modifying, and optimizing job schedules. Implementation requires
  - Inputs: Technician location, availability, skill set.
  - Processing: Optimize scheduling, dispatch jobs based on technician availability and job requirements.
  - Outputs: Technician schedule updates, job assignment notifications.
  - Related Test Cases: Verify job scheduling and assignment functionality, including optimization accuracy and real-time updates.
  - Pre-Conditions: Technicians with skill sets and jobs populated; system connected to the live database for real-time data access.
  - Expected Output: Jobs assigned to technicians based on rules; schedule updates are visible in the technician's dashboard in real-time.
  - **Tools and technologies**:
    a) Programming Languages: Java, Python, JavaScript (Node.js), or other suitable languages for backend development.
    b) Frameworks:
       i. Java: Spring Boot
       ii. Python: Django
       iii. JavaScript: Express.js
    c) Databases: PostgreSQL, MySQL, MongoDB.
    d) Frontend Libraries: React, Angular, Vue.js.

e) Mapping APIs: Google Maps API, Mapbox.

f) **Notification Services**:

i. **SMS**: Twilio

ii. **Push Notifications**: Firebase Cloud Messaging

- **Real-Time GPS Tracking:**
  - Description: The system must provide real-time GPS tracking to monitor the locations of field technicians. This functionality should enable optimal routing, job dispatching, and location-based decision-making. Integration with a GPS tracking API such as Google Maps API or Mapbox should be implemented.
  - Inputs: Technician location data from GPS devices or mobile apps.
  - Processing: Capture and process GPS location data to provide real-time visibility of technician positions. Integrate with routing algorithms to optimize job dispatch and provide location-based updates.
  - Outputs: Real-time location updates on a map, optimized routes for technicians, location-based alerts and notifications.
  - Related Test Cases: Verify accuracy of location tracking, test real-time updates and routing calculations, ensure location-based notifications are triggered correctly.
  - Pre-Conditions: GPS devices or mobile apps providing location data; integration with GPS tracking API configured; system connected to the live database.
  - Expected Output: Real-time display of technician locations on a map; accurate and optimized routing for technicians; location-based alerts and notifications visible to dispatchers and technicians.
  - Technologies and Tools:

    a) GPS Tracking APIs:

    i. Google Maps API

    ii. Mapbox

    b) Programming Languages: Java, Python, JavaScript (Node.js) for backend and frontend development.

    c) Frameworks:

    i. Java: Spring Boot

    ii. Python: Django

    iii. JavaScript: Express.js

d) Frontend Libraries: React, Angular, Vue.js (for displaying real-time maps and updates)

e) Backend Services:

i. Real-time Data Processing: Node.js, Redis (for handling live data updates)

- **Inventory Management:**
  - Description: The system must manage inventory levels and supplies used in field operations. This includes tracking inventory, updating stock levels, and providing alerts for low stock. The system should also support integration with inventory management tools and databases.
  - Inputs: Inventory data, stock levels, supply usage information, purchase orders.
  - Processing: Track inventory levels, update stock based on usage and new purchases, generate alerts for low stock, and integrate with external inventory management systems if needed.
  - Outputs: Updated inventory levels, stock alerts, inventory reports.
  - Related Test Cases: Verify inventory tracking accuracy, stock level updates, and alert generation. Test integration with external systems.
  - Pre-Conditions: Inventory database populated with initial stock levels; integration with external inventory systems configure.
  - Expected Output: Accurate inventory tracking with real-time updates; alerts for low stock; inventory reports generated as required.
  - Technologies and Tools:

    a) Databases: PostgreSQL, MySQL, MongoDB (for storing inventory data)

    b) Backend Frameworks:

    i. Java: Spring Boot

    ii. Python: Django

    iii. JavaScript: Express.js (Node.js)

    c) Frontend Libraries: React, Angular, Vue.js (for user interfaces)

    d) Integration Tools: APIs or middleware for connecting with external inventory management systems or ERP solutions

    e) Reporting Tools: Libraries or tools for generating inventory reports and visualizations (e.g., D3.js, Chart.js)

- **Invoicing and Payment Processing:**

- Description: The system must handle the generation of invoices, processing of payments, and management of financial transactions. It should support various payment methods and integrate with payment gateways for transaction processing.
- Inputs: Job details, service costs, customer payment information, invoice data.
- Processing: Generate invoices based on job details and service costs, process payments through integrated payment gateways, manage financial transactions, and update payment statuses
- Outputs: Invoices, payment confirmations, transaction records, financial reports.
- Related Test Cases: Verify invoice generation accuracy, payment processing functionality, integration with payment gateways, and financial reporting.
- Pre-Conditions: Payment gateway accounts configured; invoicing rules defined; system connected to financial databases.
- Expected Output: Accurate invoices generated and sent to customers; payments processed successfully; transaction records updated; financial reports generated as needed.
- Technologies and Tools:
  a) Payment Gateways: Stripe, PayPal, Square (for processing payments)
  b) Databases: PostgreSQL, MySQL, MongoDB (for storing financial data and transaction records)
  c) Backend Frameworks:
     i. Java: Spring Boot
     ii. Python: Django
     iii. JavaScript: Express.js (Node.js)
  d) Frontend Libraries: React, Angular, Vue.js (for user interfaces and payment forms)
  e) Reporting Tools: Libraries or tools for generating financial reports and analytics (e.g., D3.js, Chart.js)
  f) Integration Tools: APIs or middleware for connecting with payment gateways and financial systems

- **Performance Reporting:**
  - Description: The system must provide reporting tools to analyze and assess performance metrics and service outcomes. This includes generating reports on job completion times, technician efficiency, and other key performance indicators (KPIs) relevant to field service operations.

- Inputs: Job data, technician performance data, service outcomes, user feedback.
- Processing: Aggregate and analyze data to generate reports on various performance metrics. Support customization of reports based on user-defined criteria and KPIs.
- Outputs: Performance reports, charts, and dashboards showing metrics such as job completion times, technician efficiency, and service outcomes.
- Related Test Cases: Verify accuracy of report generation, data aggregation, and the ability to customize reports based on different criteria.
- Pre-Conditions: Data sources populated with performance metrics; reporting tools configured.
- Expected Output: Detailed performance reports with accurate data visualizations; ability to filter and customize reports based on user needs.
- Technologies and Tools:
  a) Reporting Libraries: D3.js, Chart.js, Highcharts (for data visualization and charting)
  b) Business Intelligence (BI) Tools: Tableau, Power BI, Looker (for advanced reporting and analytics)
  c) Databases: PostgreSQL, MySQL, MongoDB (for storing performance data)
  d) Backend Frameworks:
     i. Java: Spring Boot
     ii. Python: Django
     iii. JavaScript: Express.js (Node.js)
  e) Frontend Libraries: React, Angular, Vue.js (for displaying reports and dashboards)
  f) APIs: Integration with external BI tools or data sources if needed

- **Offline Access:**
  - Description: The system must provide offline access to critical features and data for field technicians when they are not connected to the internet. This includes the ability to view job details, update job statuses, and sync data once connectivity is restored.
  - Inputs: Job data, technician updates, offline data storage.
  - Processing: Enable data storage and functionality to be available offline, handle synchronization of offline data with the central system when the connection is reestablished.
  - Outputs: Updated job statuses, synchronized data, offline access to job details.

- Related Test Cases: Verify offline functionality, data synchronization, and error handling during connectivity loss and restoration.
- Pre-Conditions: Offline data storage mechanisms implemented; synchronization processes defined.
- Expected Output: Technicians can access and update job information while offline; data is accurately synchronized with the central system once online.
- Technologies and Tools:
  a) Local Storage Solutions: SQLite, IndexedDB (for storing data locally on devices)
  b) Offline Frameworks: PouchDB (syncs with CouchDB), Service Workers (for caching data and offline functionality)
  c) Backend Frameworks:
     i. Java: Spring Boot
     ii. Python: Django
     iii. JavaScript: Express.js (Node.js) (for defining APIs that handle synchronization)
  d) Frontend Libraries: React, Angular, Vue.js (for offline access UI/UX)

3.2. Non-Functional Requirements

- • **Performance Requirements**
  - Response Time: Job scheduling updates should be processed within 5 seconds.
  - Throughput: Support up to 500 concurrent users and handle 1000 transactions per minute.
  - Scalability: The system must be capable of vertical and horizontal scaling.

- • **Security Requirements**
  - Authentication and Authorization: Implement MFA (e.g., DUO Security) and RBAC.
  - Data Encryption: Use AES-256 for data at rest and TLS 1.2/1.3 for data in transit.
  - Data Privacy: Comply with applicable U.S. privacy laws.

- **Usability Requirements**
  - User Interface: Intuitive and user-friendly interface with responsive design.
  - Accessibility: Meet Section 508 accessibility standards.

- **Reliability Requirements**
  - Uptime: Achieve 99.9% availability.
  - Error Handling: Implement robust error handling and logging.

    - Backup and Recovery: Daily backups with an RTO of 4 hours and an RPO of 1 hour.

- **Compliance Requirements**
    - Regulatory Compliance: Adhere to OSHA and DOT regulations.
    - Standards Compliance: Follow ANSI standards relevant to field operations.

4. External Interface Requirements

4.1. User Interfaces:

- Web Application: The user interface should be accessible via popular web browsers (e.g., Chrome, Firefox, Edge) and support responsive design for various screen sizes.
- Mobile Application: The interface should be designed for iOS and Android platforms, providing a consistent user experience across different devices and screen sizes.
- Accessibility: The UI must adhere to accessibility standards such as Section 508 to ensure usability for individuals with disabilities.

4.2. Hardware Interfaces:

- GPS Devices: Integration with GPS hardware for real-time location tracking of field technicians. Ensure compatibility with standard GPS devices used in mobile and vehicle-based applications.
- Barcode Scanners: Support for barcode scanning hardware for inventory management and job tracking. Ensure compatibility with commonly used barcode scanner models.
- Printers: Integration with printers for generating physical invoices or reports. Support standard printer interfaces (e.g., USB, network printers).

4.3. Software Interfaces:

- ERP Systems: Integration with ERP systems (e.g., SAP, Oracle) for data exchange related to inventory, procurement, and financial management.
- CRM Systems: Interface with CRM systems (e.g., Salesforce, HubSpot) for customer data management and synchronization.
- Payment Gateways: Integration with payment processing services (e.g., Stripe, PayPal) for handling transactions and payments.
- Mapping APIs: Use APIs such as Google Maps API or Mapbox for location services, route optimization, and real-time tracking.

4.4. Communication Interfaces:

- APIs: Provide RESTful APIs or GraphQL endpoints for integration with third-party tools and services. Define the API endpoints, data formats (e.g., JSON, XML), and authentication methods.
- Webhooks: Support for webhooks to notify external systems of specific events or updates (e.g., job status changes, new invoices).
- Data Import/Export: Define methods for importing and exporting data in standard formats (e.g., CSV, Excel) for integration with other systems or reporting tools.

5. System Features

5.1. User Authentication and Role Management

- **Description**: Securely manage user access and roles.
- **Functionality**: Login/logout, role-based access control, user management.

5.2. Job Scheduling and Dispatch

- **Description**: Manage job schedules and dispatch tasks.
- **Functionality**: Job creation, optimization, dispatch notifications.

5.3. Real-Time GPS Tracking

- **Description**: Track technician locations in real-time.
- **Functionality**: Location tracking, route optimization, geofencing.

5.4. Inventory Management

- **Description**: Manage and track inventory and supplies.
- **Functionality**: Inventory tracking, replenishment, inspection.

5.5. Invoicing and Payment Processing

- **Description**: Handle invoicing and payments.
- **Functionality**: Invoice generation, payment processing, financial reporting.

5.6. Performance Reporting

- **Description**: Analyze and report on performance metrics.
- **Functionality**: Report generation, data analysis, dashboard.

5.7. Offline Access

- **Description**: Ensure functionality without internet connectivity.
- **Functionality**: Offline data access, synchronization, local storage.

.

6. Other Non-Functional Requirements

6.1. Performance Requirements

- Response Time: <2 seconds for user login, <5 seconds for job scheduling updates.
- Throughput: Up to 500 job dispatches/hour, 2000 concurrent users.

6.2. Security Requirements

- Authentication: Support MFA using DUO Security or similar tools.
- Data Encryption: TLS 1.2/1.3 for data in transit, AES-256 for data at rest.
- Access Control: Role-based access control (RBAC) implementation.
- Data Protection: Compliance with relevant data protection regulations.

6.3. Usability Requirements

- User Interface Design: Comply with Section 508 accessibility standards, responsive design.
- Training and Documentation: Provide user manuals and training materials.

6.4. Reliability Requirements

- Uptime: 99.9% availability.
- Fault Tolerance: Use redundant servers, data backups.
- Error Handling: Clear error messages and logging.

6.5. Compliance Requirements

- Regulatory Compliance: Adhere to OSHA and DOT regulations.
- Standards Compliance: Follow ISO/IEC 27001 and PCI-DSS (if applicable).

6.6. Maintainability Requirements

- Code Quality: Comprehensive documentation, modular design.
- Support: Support mechanisms for system issues and user inquiries.

6.7. Scalability Requirements

- System Growth: Scalable architecture for future growth.
- Performance Testing: Load testing to ensure system performance under increased load.

7. Appendices

7.1. Appendix A: Glossary:

- ERP (Enterprise Resource Planning): Software systems that manage business processes across various departments, such as inventory, accounting, human resources, etc.
- MFA (Multi-Factor Authentication): A security system that requires more than one method of authentication to verify the user's identity.
- FSM (Field Service Management): Software that manages field service operations, such as job scheduling, dispatching, and technician tracking.
- OSHA (Occupational Safety and Health Administration): A U.S. regulatory body that ensures safe and healthy working conditions by setting and enforcing standards.

- DOT (Department of Transportation): A U.S. government department responsible for regulations concerning transportation, including vehicle safety standards for field technicians.
- GPS (Global Positioning System): Satellite-based navigation that provides real-time location data for tracking technicians and assets.
- QuickBooks: A financial software used for managing accounting, invoicing, and payments.
- DUO Security: A third-party service that provides multi-factor authentication for user verification.
- Stripe: A payment processing service used to handle online financial transactions securely.
- PostgreSQL: An open-source relational database system used for managing structured data.
- Spring Boot: A Java-based framework used for creating microservices and building backend systems.
- React: A JavaScript library used for building user interfaces, especially in single-page applications.
- Firebase Cloud Messaging (FCM): A cross-platform messaging solution for sending notifications to web and mobile apps.

7.2. Appendix B: Analysis Models:

7.2.1 Use Case Diagrams

- User Authentication and Role Management: A diagram illustrating the interactions between users (technicians, managers, and administrators) and the authentication system.
- Job Scheduling and Dispatch: A diagram detailing the flow of job assignments, technician status updates, and interaction with GPS tracking.
- Invoicing and Payment: Representation of the processes involved in invoice generation, payment processing, and transaction recording.

7.2.2 Sequence Diagrams

- Real-Time GPS Tracking: Showcases the sequence of operations involved in tracking technicians and dispatching job information based on their locations.
- Inventory Management: A sequence diagram showing how inventory is updated when a technician reports tool usage, tracks item locations, or places orders for supplies.

7.2.3 Data Flow Diagrams (DFD)

- FSM System Overview: A high-level DFD showing how data flows between the FSM system, technicians, inventory management, invoicing, and third-party tools like ERP and GPS services.
- Offline Access: DFD showing the flow of data when a technician operates offline and how the system synchronizes once connectivity is restored.

7.3 Appendix C: Supporting Information:

7.3.1 OSHA Field Operations Manual (FOM) – Ensures that technician job safety requirements are met.

7.3.2 DOT Compliance Guidelines – For technician vehicle safety and route compliance.

7.3.3 U.S. Government GPS Standard Positioning Service Performance Specification – Governs GPS use for navigation and tracking in the FSM system.

7.3.4 NIST Cybersecurity Framework – Applied to ensure robust security protocols, including MFA implementation.

7.3.5 Industry Best Practices for ERP System Integration – Used for integrating ERP systems with FSM.

7.3.6 QuickBooks API Documentation – For financial and invoicing system integration.

7.4 Appendix D: Third-Party Services and Tools:

7.4.1 Google Maps API: Used for mapping and real-time GPS tracking of technicians.

7.4.2 Stripe: For secure payment processing and invoicing.

7.4.3 DUO Security: Provides MFA for the system to ensure that user access is verified securely.

7.4.4 Twilio: For SMS-based notification services for job dispatches and status updates.

7.4.5 Firebase Cloud Messaging (FCM): Push notification service to notify technicians of new jobs or updates.

7.4.6 PostgreSQL: The relational database used to manage data such as job assignments, technician details, and inventory.

7.4.7 Spring Boot: Framework used for building the backend of the FSM system, especially job scheduling and real-time updates.

7.4.8 **React**: Frontend library for building the technician dashboard, ensuring real-time display of job schedules, notifications, and updates.

7.5 Appendix E: Regulatory Compliance:

7.5.1 **OSHA Compliance**: The FSM system is designed with OSHA safety standards in mind, ensuring technicians are not assigned to jobs that would violate safety regulations.

7.5.2 **DOT Compliance**: The FSM system follows DOT vehicle safety and transport regulations, especially for technicians using company vehicles for job site visits.

7.5.3 **Data Privacy Regulations**: The system adheres to American data privacy laws such as the California Consumer Privacy Act (CCPA) to protect user and customer information.